DEVELOPMENT DIARY

Greenfield Local Hub

Tochukwu Chukwudolue

Contents

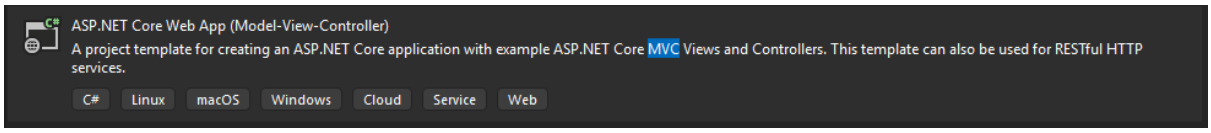
Version 1.0 2
Day 1 2
Creating and setting up the project 2
Models 3
Version 1.1 6
Day 2 6
Seeding Roles 6
Seeding Producers 7
Seeding Products 8
Basket Functionality 9
Version 1.2 10
Day 3 10
Basket Page creation and editing 10
Orders page 12
Orders Authorisation 12
Producer Dashboard 13
Version 2.0 15
Day 4 15
Gmail Login system 15
Page creation and CSS Integration 16
Extra Functionality 22
Loyalty Program 24
Accessibility Settings 25

Version 1.0

Day 1

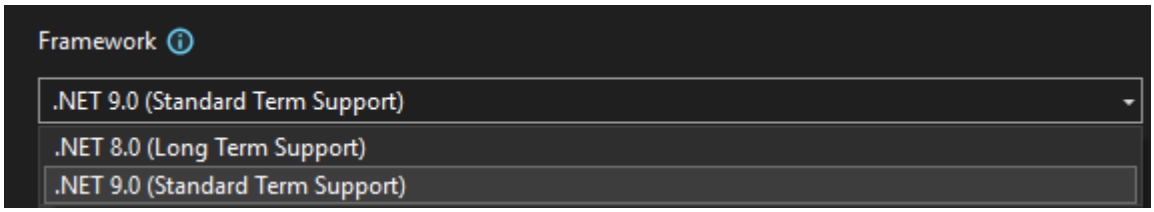
Creating and setting up the project

The project was created using ASP.NET Core Web App.



This template allows us to take advantage of Models, Views and Controllers, which are utilised when creating both the front end and back end of a web application.

We name the project E-commerce, which would be a placeholder for the folder name. We selected the .NET 9.0 Framework since it allows for better performance, new features, scalability and better optimisation.

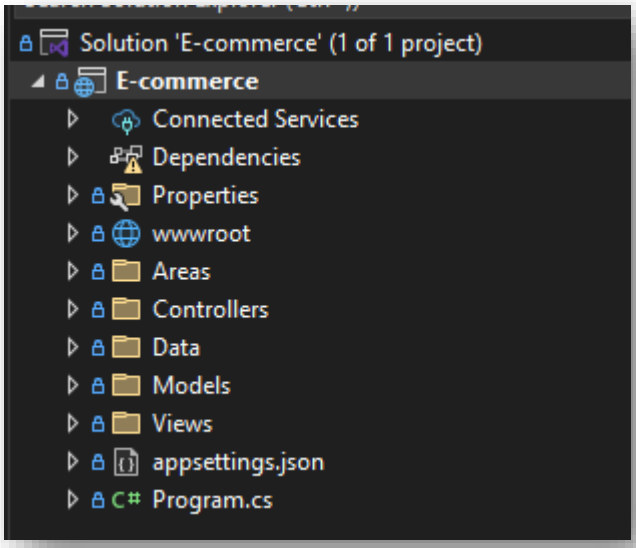


We select Individual accounts as this creates a Register and login template which we can utilise for our web application. It creates the user database for us and all the functionality, being able to edit it and customise it to our needs.

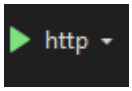
We then tap on create. This builds the template for us to work on. Everything appears as expected, but three additional packages were installed, which allowed for some extra functionality. They were;

1. Microsoft.VisualStudio. Web.CodeGeneration. Design
2. Microsoft.EntityFrameworkCore. Design
3. Microsoft. Entity. FrameworkCore.SqlServer

The solution comes with pre-installed folders, which would house the Models, Views and Controllers which would be built for the solution.



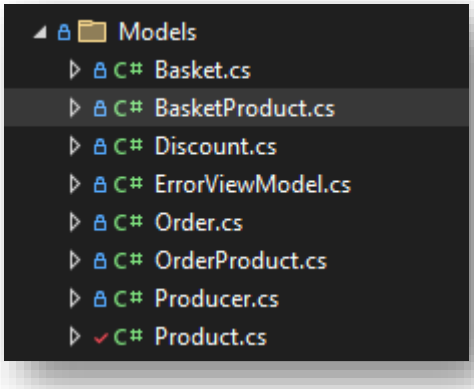
Before we click on the run button, we change the format from HTTPS to HTTP. After which, we can check whether there are any build errors in the project



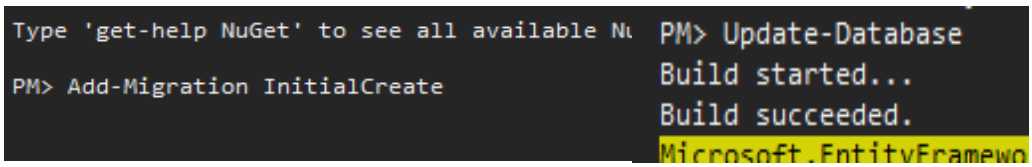
The Website appears to be functional with four buttons (Home and Privacy), alongside the Register and Log-In.

Models

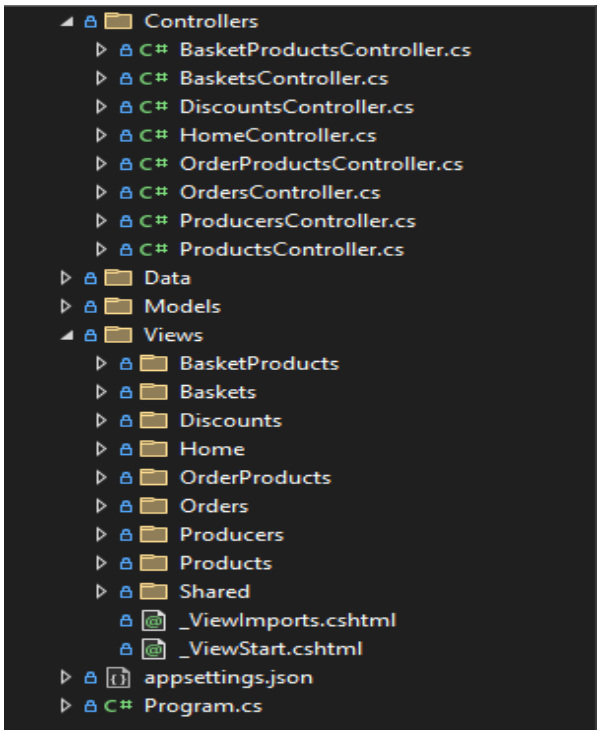
In my initial Entity Relationship diagram, I created models which I deemed appropriate and assigned attributes to each. However, upon creating the models, I noticed redundancy in my initial design and made edits to most. I added “MaxUses” and “UsedCount” to the discount model to prevent multiple usage of the same discount code, added “Delivery Type” to Order as it was a design requirement I had overseen, and added price Attributes (Unit Price and Total Price) to Order Products and Basket Products. I have also added “Status” in Basket as well as removed “Last Update” due to irrelevance, and finally, removed “Is Available” in Product due to redundancy.



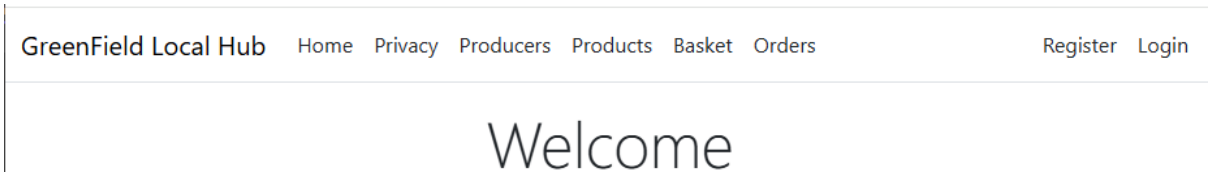
After creating the models, I added my first Migration and updated the database.



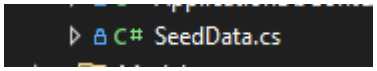
I did edit the Producer model as well, removing the “LastName” attribute and renaming the “FirstName” attribute to “ProducerName”. Next, I scaffolded my models, which ended up creating views and controllers for each respective model.



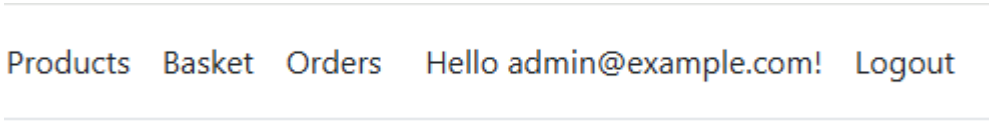
I then added navigation Links to each of these for ease of the development process, as well as added a placeholder Logo.



I then created a new file called seed data. This would essentially hold pre-placed data, which would be visible across the website.



I created a seed roles function for an admin, i.e., created an admin account.



Today

I have been able to;

- Created the web project using ASP.NET Core MVC.
- Created the models for the project, which are essentially the same in the entity relationship diagram
- Scaffolded those models which created their views and controllers
- Added appropriate migration and updated databases
- Added Navigation Links to allow me to navigate through the different webpages
- Create a seed data file and seeded Roles for an admin.

Tomorrow

I wish to;

- Finish seeding all Roles
- Seed users, i.e., producers
- Seed products
- Test for successful seeding process
- Create basic functionality
- Work on controller Logic

Version 1.1

Day 2

Seeding Roles

I resume from where I left off yesterday by seeding roles into the system. There are 3 essential things which I plan on seeding, Roles for Authentication, Products and Producers, which would be visible data seen on the website.

```
// Seed roles
string[] roleNames = { "Admin", "Producer", "Customer", "Developer" };

foreach (string roleName in roleNames)
{
    var roleExists = await roleManager.RoleExistsAsync(roleName);

    if (!roleExists)
    {
        var role = new IdentityRole(roleName);
        await roleManager.CreateAsync(role);
    }
}

// Admin
var adminUser = await userManager.FindByEmailAsync("admin@example.com");
if (adminUser == null)
{
    adminUser = new IdentityUser
    {
        UserName = "admin@example.com",
        Email = "admin@example.com",
        EmailConfirmed = true
    };

    await userManager.CreateAsync(adminUser, "Password123!");
}

if (!await userManager.IsInRoleAsync(adminUser, "Admin"))
{
    await userManager.AddToRoleAsync(adminUser, "Admin");
}
```

These users were created as temporary users to develop the website. Each of them would be able to access features which are only authorised to them. The Producer and Customer are the main users of the system; hence they would be seen the most throughout development.

```
// Producers
var producerUser = await userManager.FindByEmailAsync("producer@example.com");
if (producerUser == null)
{
    producerUser = new IdentityUser
    {
        UserName = "producer@example.com",
        Email = "producer@example.com",
        EmailConfirmed = true
    };

    await userManager.CreateAsync(producerUser, "Password123!");
}
```

This is one of the producer's Login Info.

Hello producer@example.com! Logout

```
// Customer
var normalUser = await userManager.FindByEmailAsync("user@example.com");
if (normalUser == null)
{
    normalUser = new IdentityUser
    {
        UserName = "user@example.com",
        Email = "user@example.com",
        EmailConfirmed = true
    };
```

And this is a Customer Login Info.

Hello user@example.com! Logout

Seeding Producers

There will not be only one singular producer since an industry-standard farm website should typically have a variety of producers who offer a wide range of products. I encountered errors in the code, which were quickly solved and averted. All I had to do was drop the database and re-update so that information on the website would update. I equally seeded the producer information, which would be visible on the HTML page.

```
var producers = new List<Producer>
{
    new Producer
    {
        ProducerName = "Daniel's Farmfoods",
        PhoneNumber = "07700900001",
        ProductDescription = "We grow a wide variety of seasonal fruits and vegetables using traditional organic methods...",
        Location = "Birmingham",
        ProducerInfo = "Daniel's Farm has been family-owned and operated for over 30 years...",
        DateJoined = new DateOnly(2023, 4, 10),
        IsVerified = true,
        UserId = producerUser1.Id
    },
    new Producer
    {
        ProducerName = "Green Valley Gardens",
        PhoneNumber = "07700900002",
        ProductDescription = "Green Valley Gardens specialises in freshly cut herbs...",
        Location = "Coventry",
        ProducerInfo = "Green Valley Gardens was established in 2018...",
        DateJoined = new DateOnly(2023, 7, 22),
        IsVerified = true,
        UserId = producerUser2.Id
    },
    new Producer
    {
        ProducerName = "Sunny Fields Farm",
        PhoneNumber = "07700900003",
        ProductDescription = "Sunny Fields Farm produces free-range eggs...",
        Location = "Wolverhampton",
        ProducerInfo = "Sunny Fields Farm has been in the Holloway family...",
        DateJoined = new DateOnly(2024, 1, 5),
        IsVerified = true,
        UserId = producerUser3.Id
    }
```

This stage took some trial and error due to a typo in one of the models. This was solved by editing the model, adding migration, dropping the previous database and updating the database so that the system would have correct data which is correct.

ProducerName	PhoneNumber	ProductDescription	Location	ProducerInfo	DateJoined	IsVerified	
Daniel's Farmfoods	07700900001	We grow a wide variety of seasonal fruits and vegetables using traditional organic methods...	Birmingham	Daniel's Farm has been family-owned and operated for over 30 years...	10/04/2023	☑	Edit | Details | Delete
Green Valley Gardens	07700900002	Green Valley Gardens specialises in freshly cut herbs...	Coventry	Green Valley Gardens was established in 2018...	22/07/2023	☑	Edit | Details | Delete
Sunny Fields Farm	07700900003	Sunny Fields Farm produces free-range eggs...	Wolverhampton	Sunny Fields Farm has been in the Holloway family...	05/01/2024	☑	Edit | Details | Delete

These are the producers that are going to be visible on the website.

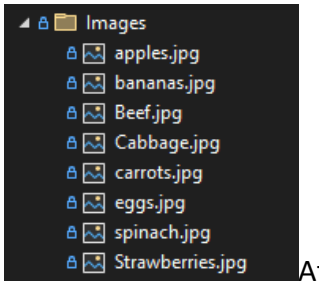
Seeding Products

Next thing that I'll be seeding is Products. These are going to be what customers purchase when they interact with the website.

```
var products = new List<Product>
{
    new Product
    {
        ProductName = "Apples",
        Description = "Fresh, crisp organic apples harvested at peak ripenes
        Price = 1.50m,
        StockQuantity = 100,
        Category = "Fruit",
        ImagePath = "/Images/apples.jpg",
        ProducerId = danielsFarmfoods.ProducerId,
        DateAdded = DateOnly.FromDateTime(DateTime.Now),
    },
    new Product
    {
        ProductName = "Carrots",
        Description = "Sweet and crunchy organic carrots.",
        Price = 0.90m,
        StockQuantity = 120,
        Category = "Vegetables",
        ImagePath = "/Images/carrots.jpg",
        ProducerId = danielsFarmfoods.ProducerId,
        DateAdded = DateOnly.FromDateTime(DateTime.Now),
    },
    new Product
    {
        ProductName = "Cabbage",
        Description = "Firm, leafy green cabbage.",
        Price = 1.20m,
        StockQuantity = 80,
        Category = "Vegetables",
        ImagePath = "/Images/Cabbage.jpg",
        ProducerId = danielsFarmfoods.ProducerId,
        DateAdded = DateOnly.FromDateTime(DateTime.Now),
found
```

Before I completed this, I had to make a separate folder for the product images on the website.

These are the few products which would be visible in the prototype. More may be added in future if the need arises.



After seeding all these, I edited the program.cs file so that the system calls the data which was planted in the backend.

ProductName	Description	ImagePath	Price	StockQuantity	Category	DateAdded	Producer
Apples	Fresh, crisp organic apples harvested at peak ripeness.	/Images/apples.jpg	1.50	97	Fruit	13/04/2026	1

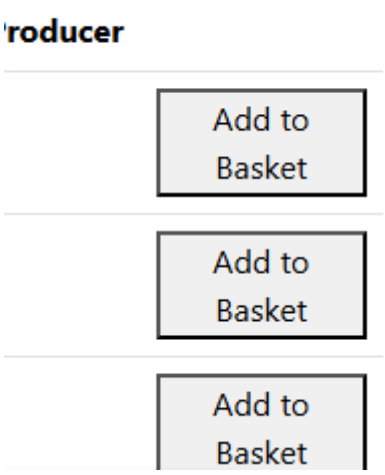
The images are not visible as of now simply because they have not been coded. This will be done at a later time.

Basket Functionality

Next is the controller Logic and basic functionality. Firstly, I'd want to find a way to add products to my basket. I'd need to add an "Add to Basket" button, which essentially would add products to my basket.

```
<form asp-controller="BasketProducts" asp-action="Create" method="post">
    <input type="hidden" name="ProductsId" value="@item.ProductId" />

    @if (item.StockQuantity > 0)
    {
        <button type="submit">Add to Basket</button>
    }
    else
    {
        <button type="button" disabled>Out of Stock</button>
    }
</form>
```

This was the form which would create a temporary Add to basket button.



They ended up looking something like this. The controller was then edited so as to redirect the user to the basket page whenever they add a product to their basket, as long as the user is logged in. If the user is not logged in, they can not access the basket.

Today

I have been able to;

- Finish seeding all Roles
- Seed users, i.e., producers
- Seed products
- Test for successful seeding process
- Work on basket functionality partially by creating an add to basket button
- Edit the controller logic in redirecting a user to the basket page.

Tomorrow

I wish to;

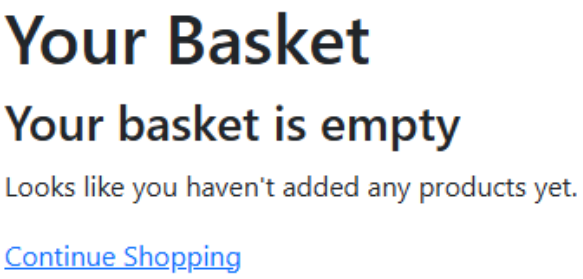
- Create and edit the basket page
- Create a checkout system
- Create a place order button which shows the user their orders on the orders page.
- Work on the orders page
- Work on Authorisation
- Create a producer dashboard.

Version 1.2

Day 3

Basket Page creation and editing

Resuming from where I had left off on the previous day, I first temporary improve the look of the basket with basic HTML. Alongside that, I edit the controller. The system scans the basket of the logged-in user. If it is empty, a message is displayed, and a navigation link to the products page is present. Otherwise, the quantity and price of a product are displayed.



This is what the basket page looks like when there is nothing inside the basket.

Your Basket

Apples

Price: £1.50

Qty: 1

Total: £1.50

Remove

Order Summary

Subtotal: £1.50
Loyalty Discount: £0.00

Total: £1.50

And this is what it looks like with an item in the basket

Checkout system

Next thing that I'll be working on is the proceed the check out button.

```
<div>
    <a asp-controller="Orders"
       asp-action="Create"
       asp-route-basketId="@basketId"
       class="btn btn-primary w-100 mb-2">
       Proceed to Checkout
    </a>
    <a asp-controller="Products"
      asp-action="Index">
       Continue Shopping
    </a>
</div>
```

Proceed to Checkout

Equally, I edited the create basket form, hence when the proceed to checkout button is tapped, the user is redirected to a checkout page.

Checkout

Order Method

○ Delivery
○ Collection

{{-- Delivery options --}} {{-- Collection date --}}

Place Order Back to Basket

© 2026 - E_commerce - Privacy

Orders page

Next is to get the place order button working. Inside the controller, the system would find the currently logged-in user and assign values for the order of that specific user. The system then sets the tracking status to pending by default. Additionally, worked on the subtotal as well as the discount calculator. The system would also have two different delivery options, which would be “Delivery” or “Collection”. With every order made, the stock of that Item is updated.

Index

Create New

OrderDate	OrderStatus	TotalAmount	DeliveryAddress	ShippingFee	DeliveryType	ScheduleDate
14/04/2026 12:14:21	Pending	11.19		0.00		18/04/2026 00:00:00
17/04/2026 10:44:50	Pending	11.49	B784WE	9.99	Next Day	

After designing the controller Logic, the next thing to do was to edit the order Index so that details of the order made are fully displayed. All products in that order, Total, Status, Date and time of when the order was made, etc. Additionally, a navigation link was placed, which would take the user back to the orders page.

Order Details

Order Date: 2026-04-17 10:44

Status: Pending

Subtotal: 11.49

Order Method: Delivery

Delivery Type: Next Day

Delivery Address: B784WE

Shipping Fee: 9.99

Products Ordered

Product	Price	Quantity	Line Total
Apples	1.50	1	1.50

Back to List

Orders Authorisation

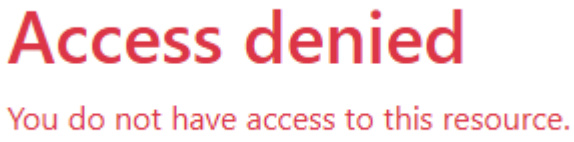
Next is Authorisation. Users should only be allowed to see their own order history and no one else’s.

```
public async Task<ActionResult> Index()
{
    var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);

    if (userId == null)
    {
        return Unauthorized();
    }
```

Producers and Admins, however, should have full access to the orders. The controller was edited to allow users of higher authority to access all orders. Producers would view these orders and start creating them. Admins would oversee all that happens.

Producer Dashboard

I created a new .cshtml file as well as a new controller dashboard. I added authorisation to allow only the producer, admin and dev to access the dashboard. Regular customers would have no way of accessing the page unless they had the login credentials of a higher authority.



.

The dashboard was edited to display basic stock information, Orders, and update products, and also made it so that producers can only edit their own products and not the products of any other producer.

```
0 references
public async Task<IActionResult> Index()
{
    var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);

    var producer = await _context.Producer
        .FirstOrDefaultAsync(p => p.UserId == userId);

    if (producer == null)
    {
        return NotFound();
    }

    var products = await _context.Product
        .Where(p => p.ProducerId == producer.ProducerId)
        .ToListAsync();

    var orders = await _context.Order.Include(o => o.OrderProducts)
        .ThenInclude(op => op.Product)
        .Where(o => o.OrderProducts
            .Any(op => op.Product != null && op.Product.ProducerId == producer.ProducerId))
        .ToListAsync();
```

Supplier Dashboard

Total Products: 4

Low Stock Products: 0

Total Stock Units: 343

Product	Price	Stock
Apples	1.50	96
Carrots	0.90	120
Cabbage	1.20	78
Beef (500g)	6.99	49

View all your products

Recent Orders

Order ID	Order Date	Status	Total
4	2026-04-14	Pending	11.19
5	2026-04-17	Pending	11.49

View All your orders

Today

I have been able to;

- Create and edit the basket page
- Create a checkout system
- Create a place order button which shows the user their orders on the orders page.
- Work on the orders page
- Work on Authorisation
- Create a producer dashboard.

Tommorrow

I want to;

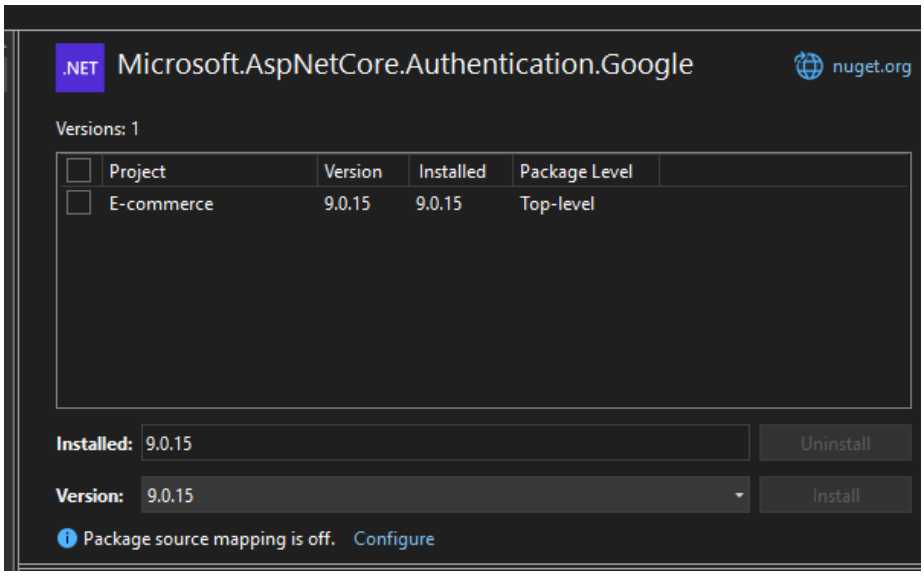
- Integrate a GMAIL Login system
- Begin adding CSS and JavaScript to the website
- Add other little bits of extra functionality
- Work on Accessibility settings
- Design the Loyalty program

Version 2.0

Day 4

Gmail Login system

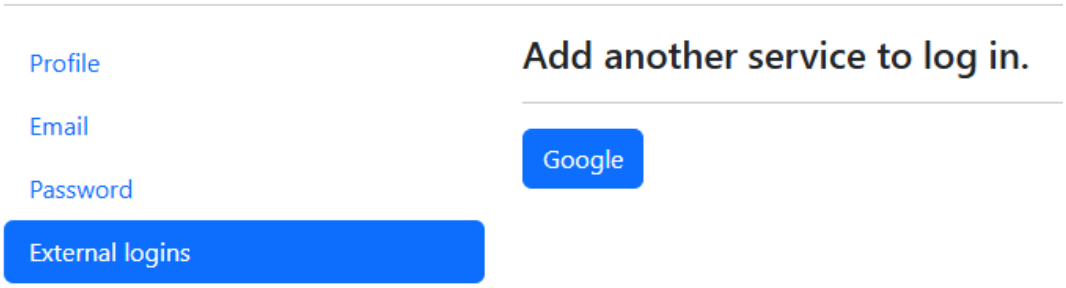
To integrate the system, we need to head over to Google Developer Console, log in with my Google Account and create a new project. After which, I would add a new credential, which is inside the APIs and Services menu. From this, I would select my client and complete the form, which essentially requires me to fill in information about the project. After which, my client ID and secret are generated. I would then go back to Visual Studio and install an authorisation package.



I then go to my appsettings.json file and import my client ID and secret.

```
"Google": {
  "ClientId": "YOUR_GOOGLE_CLIENT_ID",
  "ClientSecret": "YOUR_GOOGLE_CLIENT_SECRET"
```

When we run the application, we can see that we can log in with a Google account.

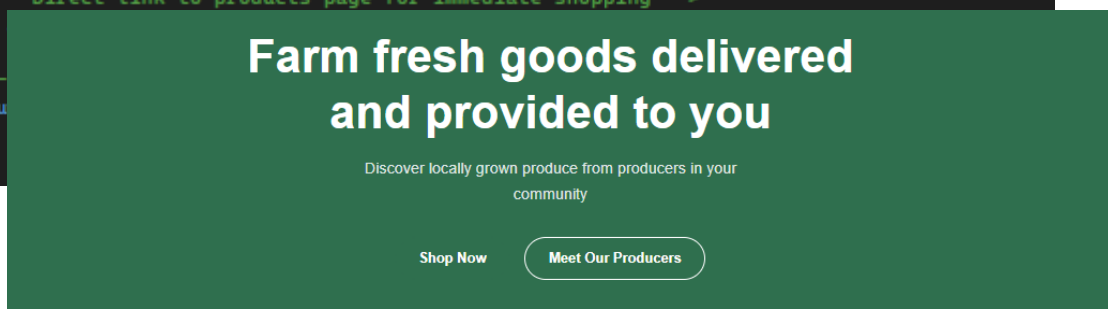


Page creation and CSS Integration

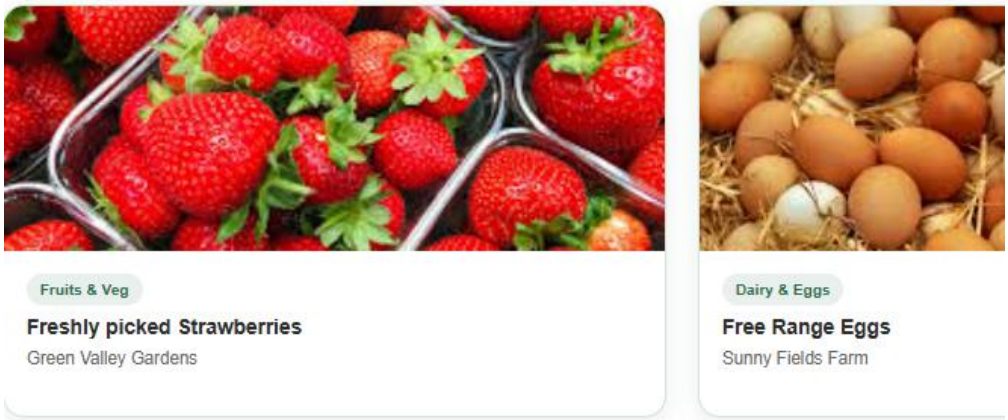
Home Page

The Home page was designed and styled using CSS in line with the Figma design mockup and the style guide. The page uses the Inter font family throughout at varying weights (400, 500, 600, 700). A hero section was created featuring a headline, subtitle and two call-to-action buttons. “Shop Now” and “Meet Our Producers”.

```
<section class="hero fade-in">
    <h1>Farm fresh goods delivered and provided to you</h1>
    <p>Discover locally grown produce from producers in your community</p>
    <div class="hero-buttons">
        <!-- Direct link to products page for immediate shopping -->
        <a

        <!-
        <bu
    </div>
</section>
```

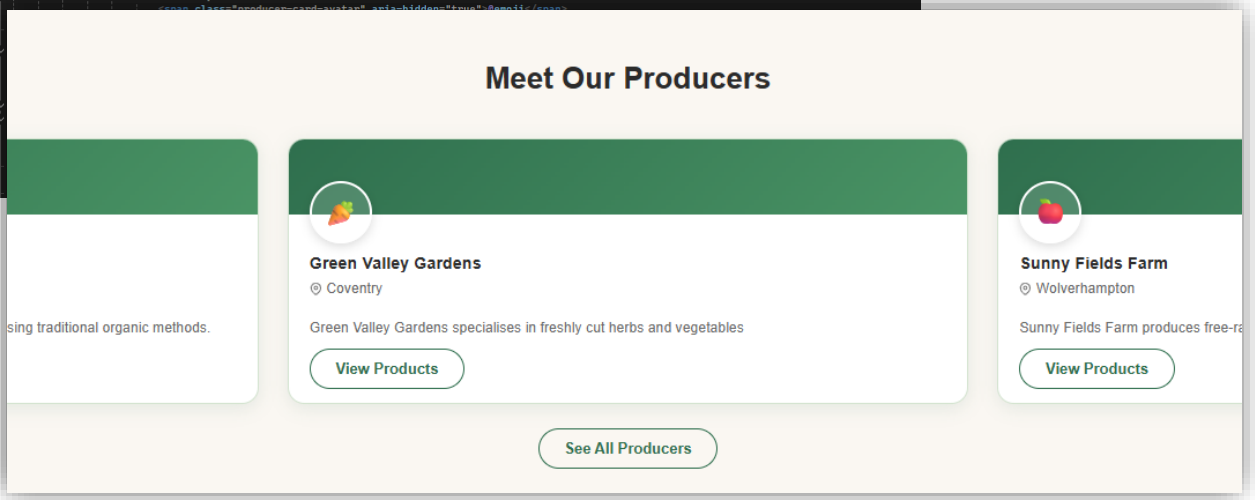


A “Fresh Picks” section was added beneath the hero to display featured product cards dynamically pulled from the database. Each card shows a category tag, product name as well as the producer’s name.



JavaScript was equally Implemented to card a hover animation; this improves depth as well as customer experience A “Meet Our Producers” section was added below this, displaying producer cards with their names and descriptions. Clicking a card’s View Products button

```
    The producerId is passed as a query parameter to filter products by producer
  -->
<section class="section fade-in" id="producers-section">
    <h2 class="section-title">Meet Our Producers</h2>
    <div class="producers-grid">
        @if (producers.Any())
        {
            int idx = 0;  // Counter for cycling through avatar emojis

            @foreach (var producer in producers)
            {
                // Cycle through emoji array using modulo operator
                // This ensures we never go out of bounds and get variety
                string emoji = avatarEmojis[idx % avatarEmojis.Length];
                idx++;

                <div class="producer-card">
                    <!-- Producer banner with emoji avatar -->
                    <div class="producer-card-banner">
```



routes to the Products page filtered by that producer.

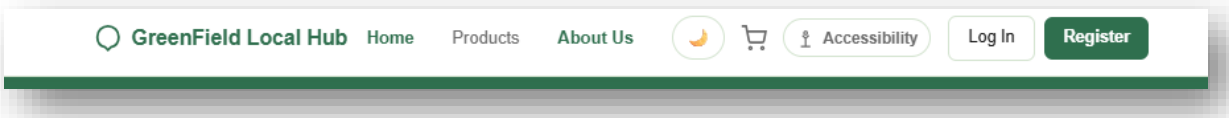
Light and dark mode CSS variables were applied globally, so all colours switch correctly when the theme is toggled. The button is located on the navigation bar.

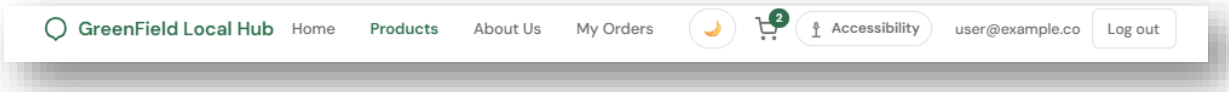
```
<script>
    // Theme toggle - reads saved preference from localStorage on load, toggles between light/dark
    const themeToggleBtn = document.getElementById('theme-toggle');
    const htmlEl = document.documentElement;
    const savedTheme = localStorage.getItem('glh-theme') || 'light';
    htmlEl.setAttribute('data-theme', savedTheme);

    function updateToggleIcon(theme) {
        if (themeToggleBtn) {
            themeToggleBtn.textContent = theme === 'dark' ? '☀' : '🌙';
            themeToggleBtn.setAttribute('aria-label', theme === 'dark' ? 'Switch to light mode' : 'Switch to dark mode');
        }
    }
    updateToggleIcon(savedTheme);

    if (themeToggleBtn) {
        themeToggleBtn.addEventListener('click', () => {
            const current = htmlEl.getAttribute('data-theme');
            const next = current === 'light' ? 'dark' : 'light';
            htmlEl.setAttribute('data-theme', next);
            localStorage.setItem('glh-theme', next);
            updateToggleIcon(next);
        });
```

Navigation Bar

```
/* — Navbar ————————————————————————
.navbar {
    background: var(--surface);
    border-bottom: 1px solid var(--border)
    padding: 0;
    position: sticky;
    top: 0;
    z-index: 1000;
    box-shadow: var(--shadow-sm);
}

.navbar-container {
    max-width: 1280px;
    margin: 0 auto;
    padding: 0 24px;
    display: flex;
    align-items: center;
    justify-content: space-between;
    height: 64px;
}

.navbar-brand {
    display: flex;
    align-items: center;
    gap: 8px;
    font-weight: 700;
    font-size: 1.1rem;
    color: var(--accent);
    text-decoration: none;
}

.navbar-toggle {
    display: none;
    flex-direction: column;
    justify-content: space-between;
    width: 30px;
    height: 21px;
    background: transparent;
```

The navigation bar was fully styled to match the Figma design. It was given sticky positioning so it remains visible at the top of the screen as the user scrolls down the page. The GLH logo was placed on the left and links to the home page when clicked. Navigation links (Home, Products, Our Producers, About Us, My Orders, Dashboard) were styled and given role-based visibility rules. My Orders is hidden from guests, and Dashboard is hidden from customers. A dark/light mode toggle button was placed in the top right. A basket icon with a live badge counter was added that updates whenever an item is added or removed. An account icon was also added, which displays login and register options for guests or account management options for authenticated users. Hover states were applied using the defined colour variables from the style guide.



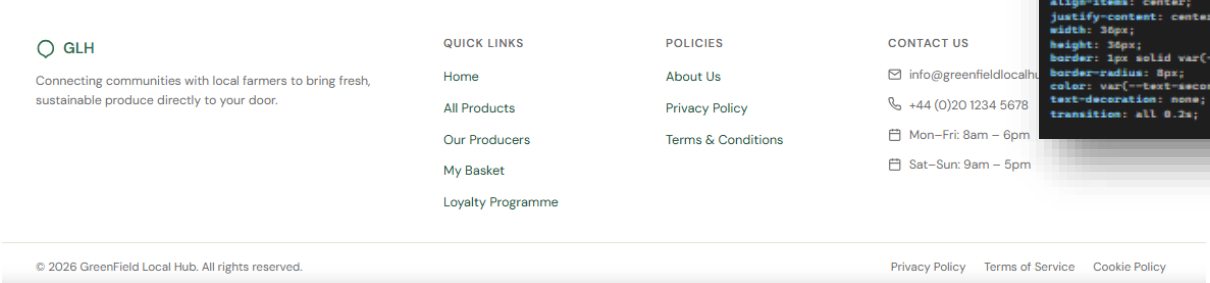


Footer

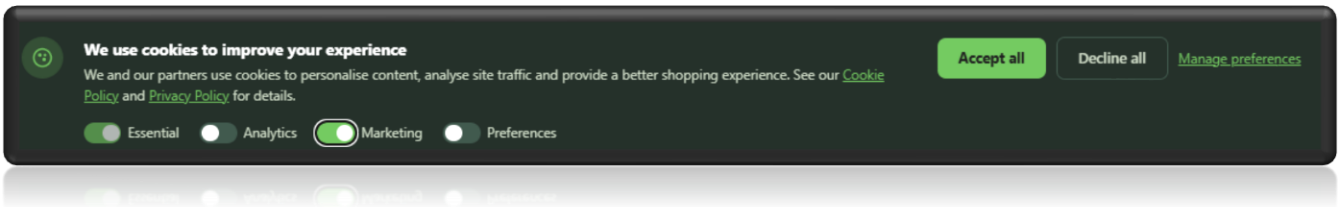
The footer was created and added to the shared layout so it appears on every page. It includes the GLH logo, which links back to the home page, quick navigation links to key pages (Products, About Us, Privacy Policy, Policies)

```html
    </a>
    <p class="footer-brand__tagline">Connecting communities with local farmers to bring fresh, sustai

<div class="footer-col">
    <h3 class="footer-col__title">Quick Links</h3>
    <ul class="footer-links" role="list">
        <li><a asp-controller="Home" asp-action="Index">Home</a></li>
        <li><a asp-controller="Products" asp-action="Index">All Products</a></li>
        <li><a asp-controller="Producers" asp-action="Index">Our Producers</a></li>
        <li><a asp-controller="Baskets" asp-action="Index">My Basket</a></li>

        <li><a asp-controller="Home" asp-action="Loyalty">Loyalty Programme</a></li>
    </ul>
</div>
<div class="footer-col">
    <h3 class="footer-col__title">Policies</h3>
    <ul class="footer-links" role="list">

        <li><a asp-controller="Home" asp-action="About">About Us</a></li>
        <li><a asp-controller="Home" asp-action="Privacy">Privacy Policy</a></li>
        <li><a asp-controller="Home" asp-action="TermsAndConditions">Terms &amp; Conditions</a></li>
    </ul>
```

```css
/* FOOTER */
site-footer {
  background: var(--surface, #fff);
  border-top: 1px solid var(--border, #e
  margin-top: auto;

ooter-container {
  max-width: 1280px;
  margin: 0 auto;
  padding: 48px 24px 32px;

ooter-grid {
  display: grid;
  grid-template-columns: 2fr 1fr 1fr 1.5
  gap: 40px;

ooter-brand__logo {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  font-weight: 700;
  font-size: 1.05rem;
  color: var(--accent, #2F6F4E);
  text-decoration: none;
  margin-bottom: 12px;

ooter-brand__tagline {
  font-size: 0.85rem;
  color: var(--text-secondary, #6b6b6b);
  line-height: 1.6;
  margin-bottom: 16px;

ooter-social {
  display: flex;
  gap: 10px;

ooter-social__link {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 36px;
  height: 36px;
  border: 1px solid var(--border, #e5e7eb
  border-radius: 8px;
  color: var(--text-secondary, #6b6b6b);
  text-decoration: none;
  transition: all 0.2s;
```

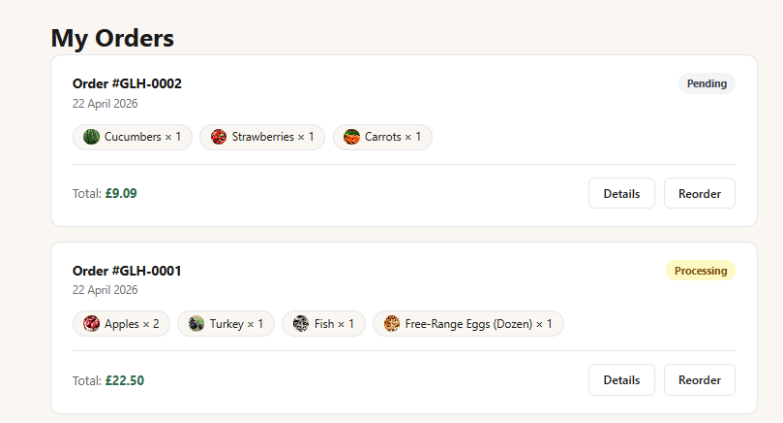


Hover effects were applied to the footer links so they change from the lighter font colour to the normal font colour on hover, as defined in the style guide. All links in the footer were verified to open the correct pages. Social media icons were configured to open in a new tab. The footer is fully responsive and adapts to smaller screen sizes correctly i.e., it boasts a responsive design. The cookies banner additionally was added at the footer as it is industry standard.



Orders

The My Orders page was styled so that each order is presented as a card. The card displays



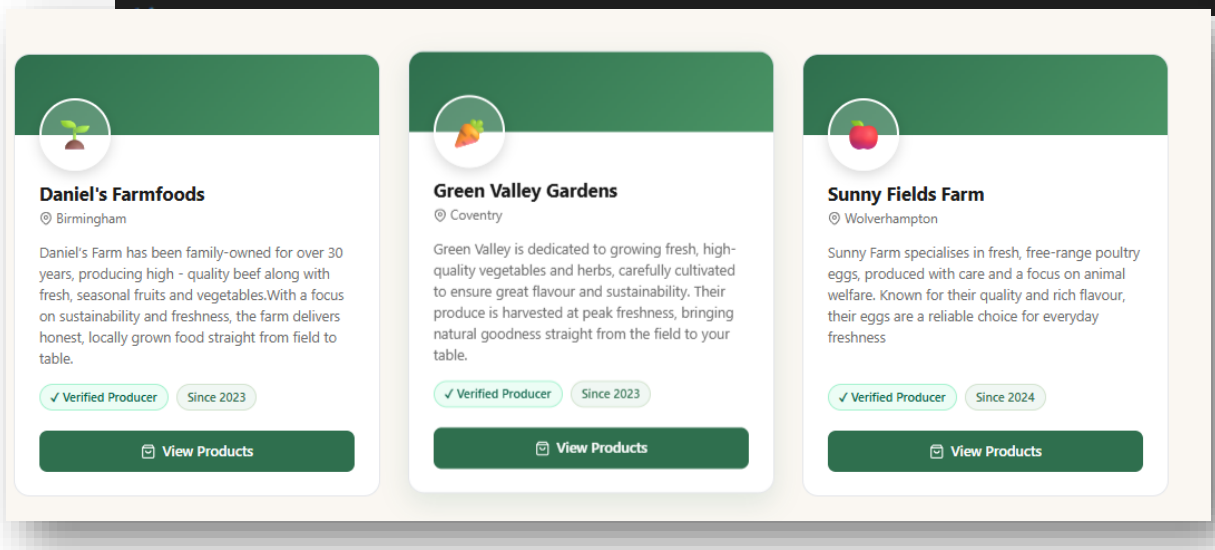
the order ID, date, a list of item chips and the order total. Orders are sorted with the most recent at the top. Status badges were implemented using colour-coded styling. Delivered in Success green (#4CAF50), Processing in Warning amber (#E9B44C), and shipped in blue (#3498db). A Reorder button was also added to each card, allowing customers to quickly re-add all items from a past order directly back into their basket.

Producers

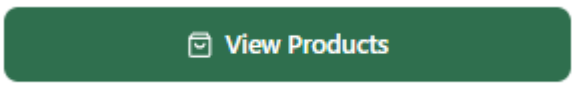
The Our Producers page was styled to display producer cards in a grid layout. Each card shows the producer's business name, a description and a View Products button. To view the producers page, the user would have to scroll down to the lower body section and tap on the "see all producers button"

```
<!-- "See All" link to full producers listing page -->
<div style="text-align:center; margin-top:32px;">
    <a asp-controller="Producers" asp-action="Index" class="btn-outline">See All Producers</a>
</div>
```

.



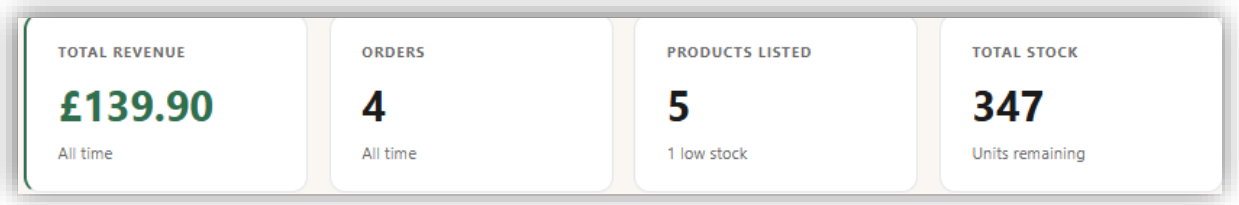
Cards were given a 30px border-radius and a drop shadow in line with the design specification. The "View Products" button routes to the Products page filtered to show only that producer's products. Producer data is loaded dynamically from the Producers database

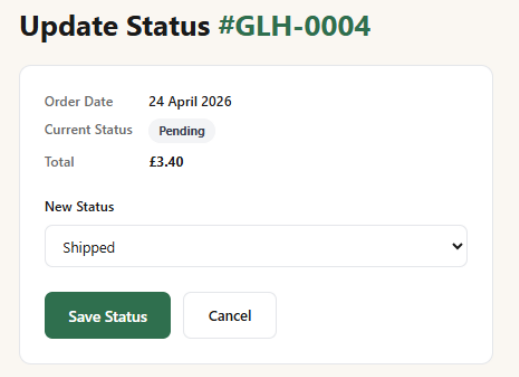


table, so the page updates automatically when producers are added or modified

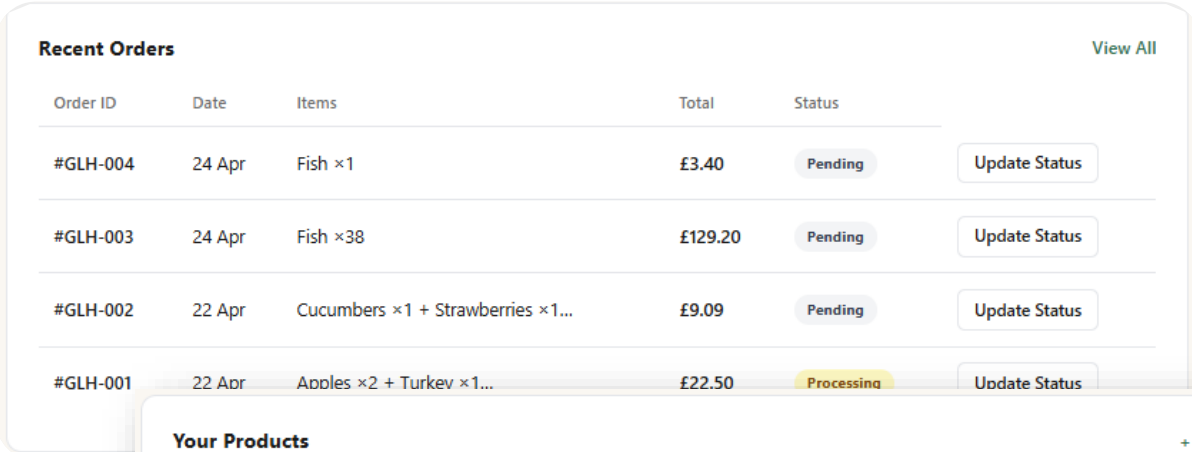
Producer Dashboard

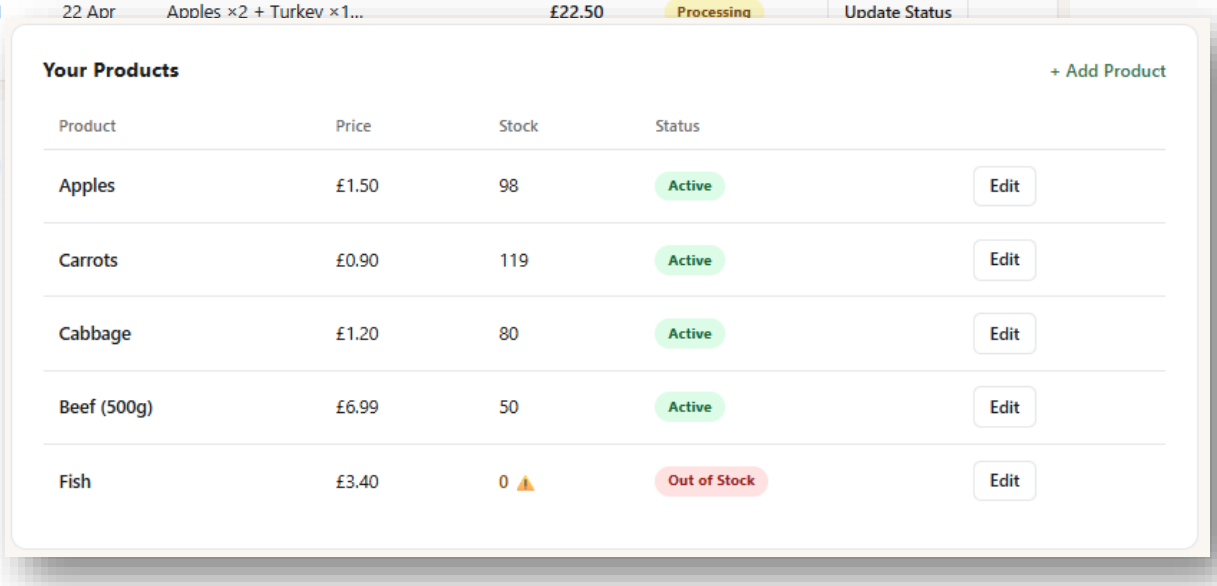
The Producer Dashboard was styled to provide a clean, professional overview interface. Four stats cards were added at the top, displaying Total Revenue, Total Orders, Products Listed and Total stock available.





which are all populated with live data from the database. A Recent Orders table was added below, showing order ID, customer name (anonymised), items ordered, total and a colour-coded status badge. Product status can also be updated with the status update button. The Stock, Pricing and Orders tabs were each given dedicated styled sub-views. Stock badges correctly display In Stock (green, >20 units), Low Stock (amber, <20 units) and Out of Stock (red, 0 units). The entire dashboard was locked behind role-based authorisation, so only Producers and Admins can access it.

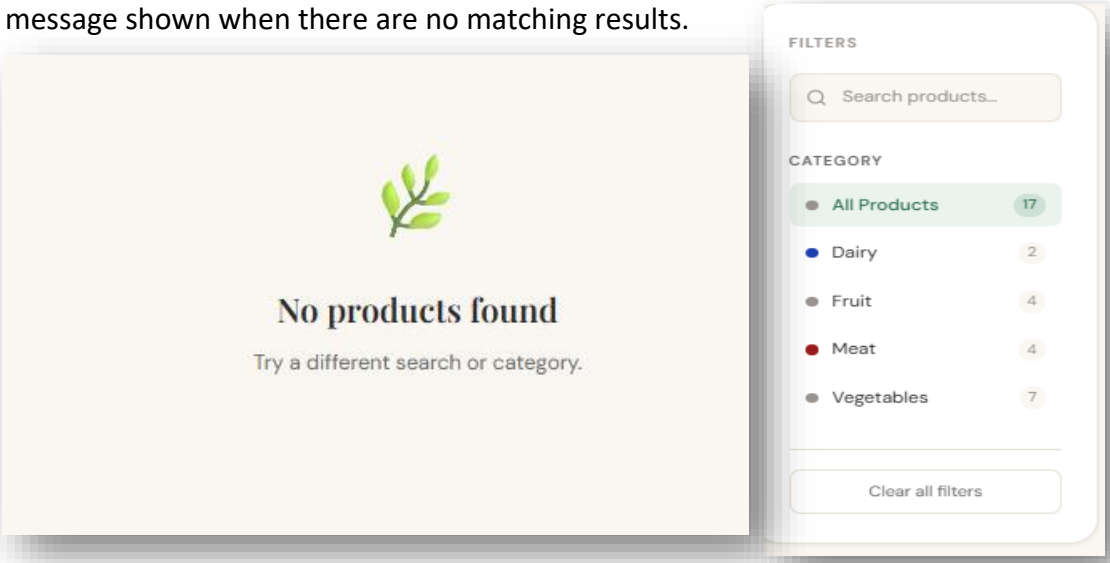




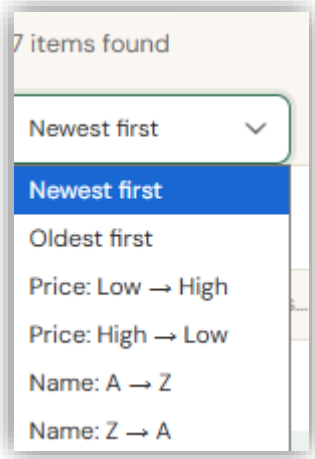
Extra Functionality

Products

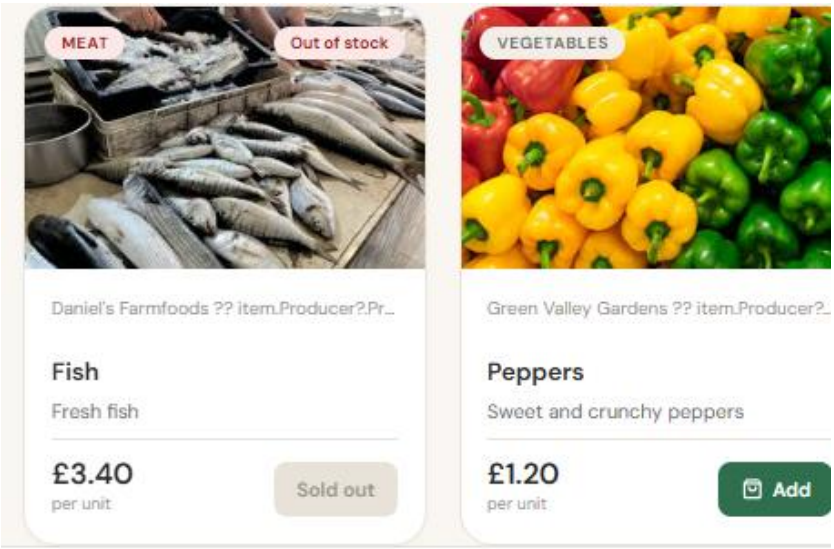
Additional functionality was added to the Products page. A live search bar was implemented so the product grid filters in real time as the user types, with a “No products found” message shown when there are no matching results.



Category filter checkboxes were added so users can filter products by category, with multiple categories selectable at once. A sort dropdown was added with options for Newest, Price: Low to High, Price: High to Low, and Rating.



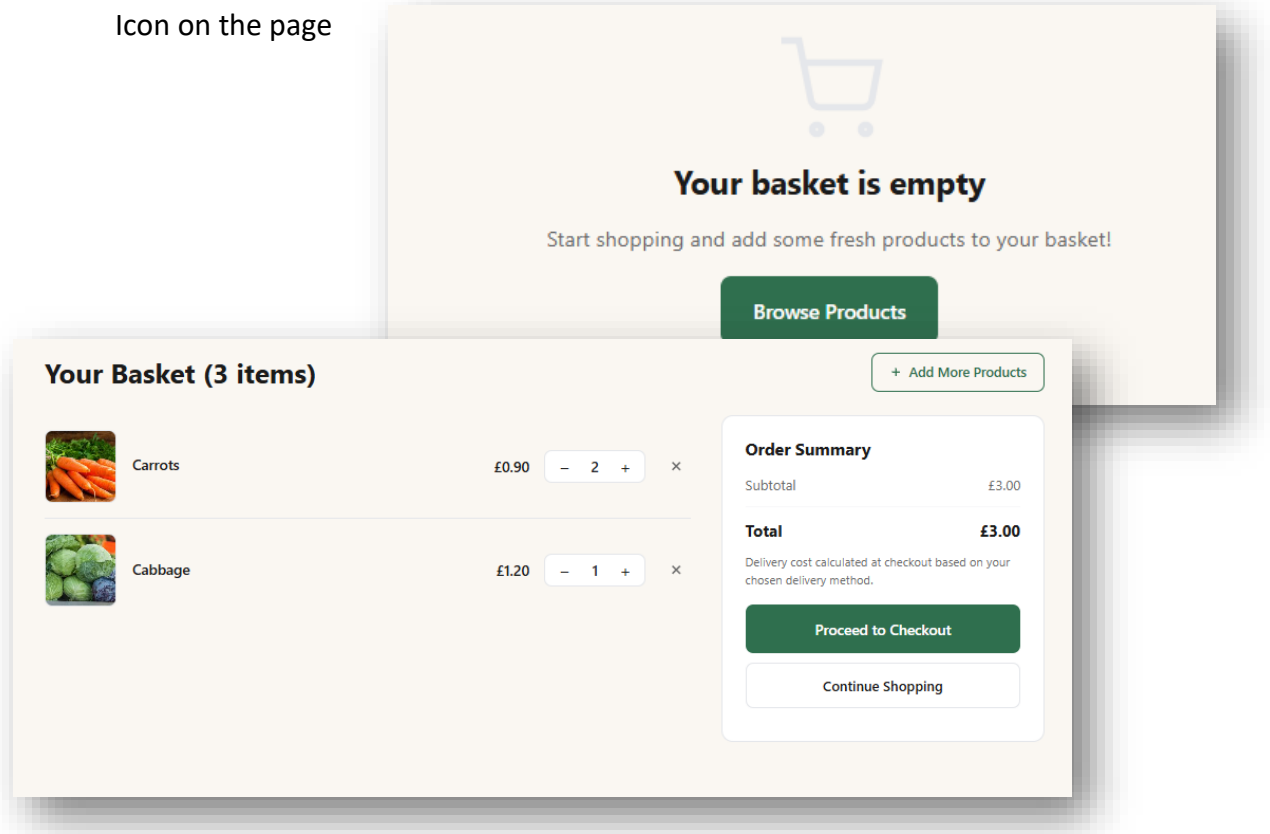
Out-of-stock products now display an “Out of Stock” badge, and the Add to Basket button is disabled for those items. Product images were coded and are now correctly displayed on each product card. The code across the Products controller and views was also commented throughout for clarity.



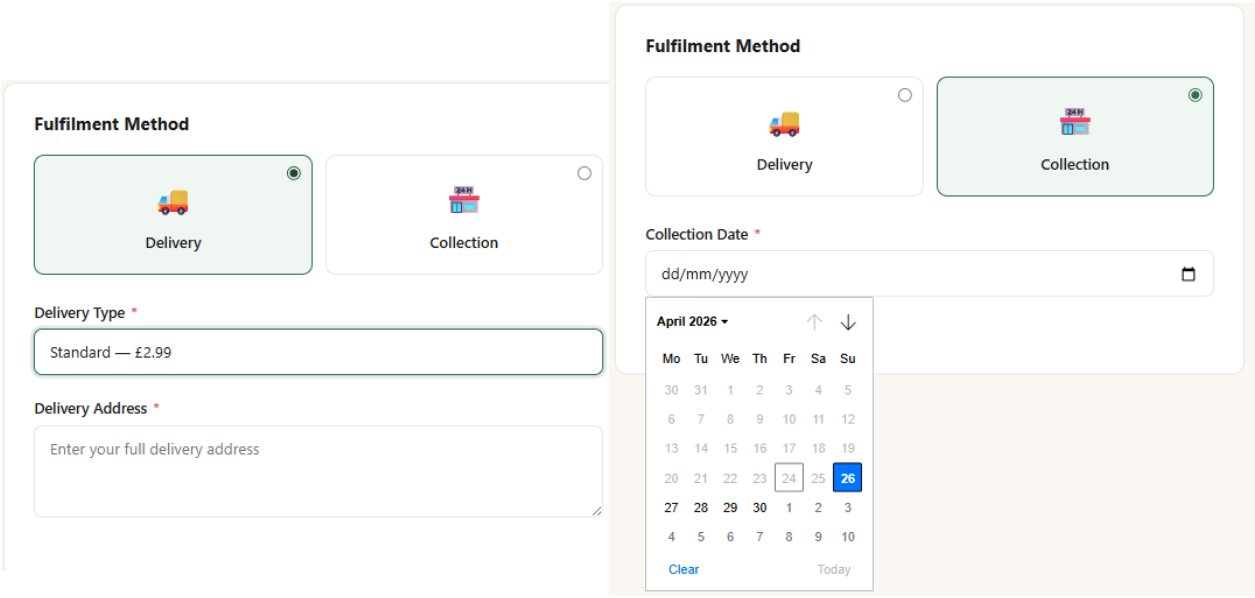
Basket

Additional basket functionality was completed. Quantity increment and decrement buttons (+ and −) were added to each basket item so users can adjust the quantity directly from the basket page. Decrementing a quantity to zero automatically removes the item from the basket. A remove button (x) was also added to each basket item for direct removal. The basket Icon on the navigation bar additionally updates with addition or removal of products.

The Order Summary panel was implemented to show the subtotal, updating in real time as quantities change. A Continue Shopping button was added that routes back to the Products page without clearing the basket. Comments were added throughout the basket controller and views to document the logic clearly. When the basket is empty, a browse button page is customised with a basket Icon on the page

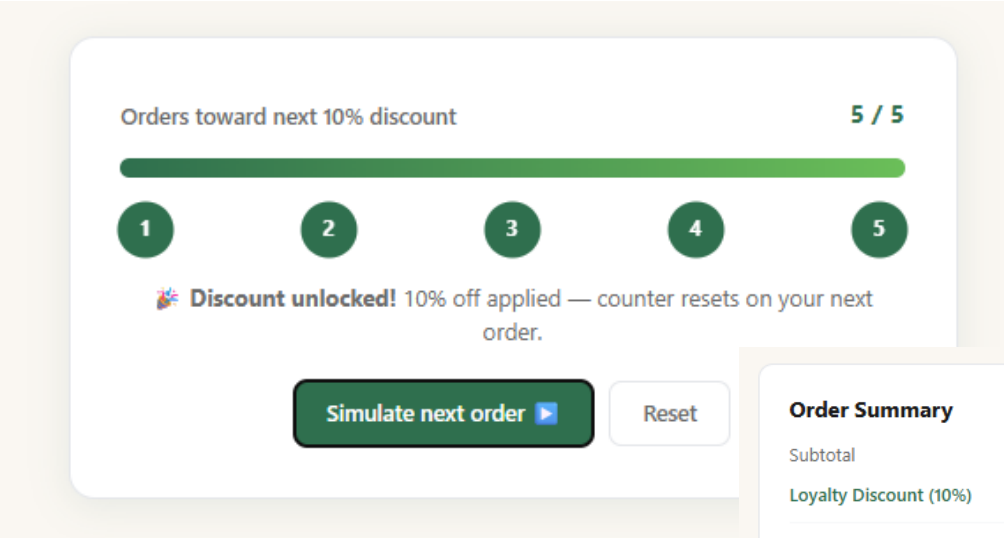


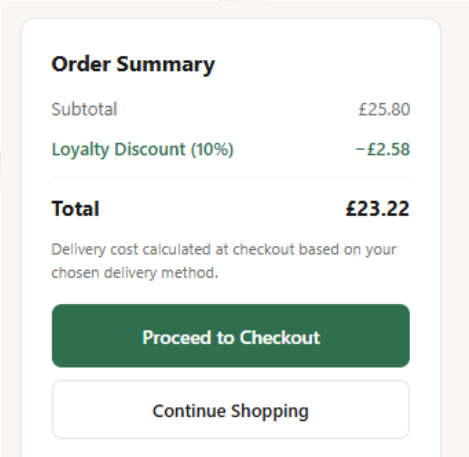
The checkout system has equally been implemented in this system. The user could choose for either delivery or collection, allowing for convenience and improved user experience.



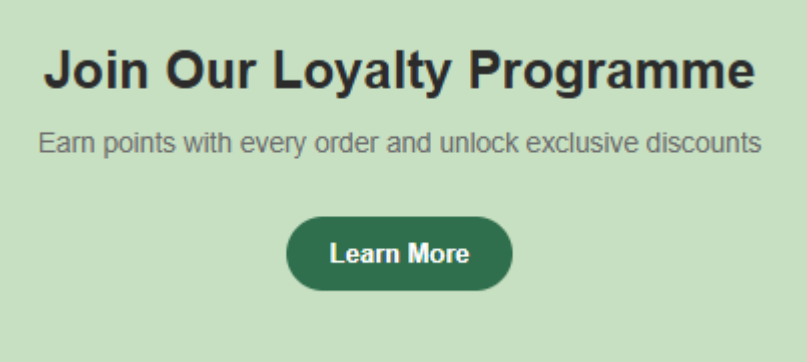
Loyalty Program
Loyalty Page

A Loyalty page was designed and built to explain the Greenfield Local Hub loyalty programme to customers. The page outlines how the scheme works, the benefits of membership and the terms and conditions associated with discount system.



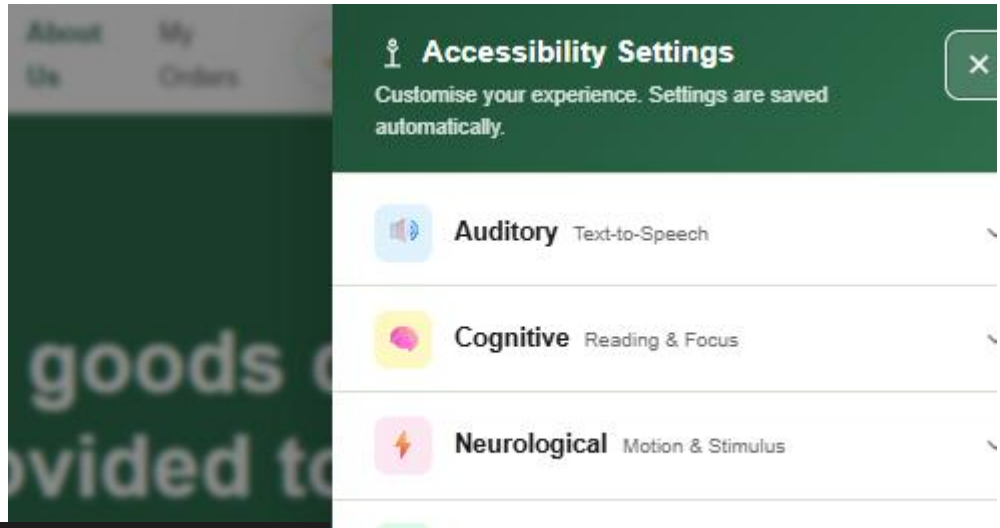


The user creates an order for a minimum of 5 orders. After the 5th order is made, they are entitled to a 10% discount which is applied at the end of their checkout process the basket checkout process was updated to allow customers to enter a discount code, which is validated against the database and applied as a line item in the Order Summary if valid. The loyalty page was linked in both the navigation bar and the footer. There is also a frequently asked questions centre where the user can reference to in case more enquiries would be needed



Accessibility Settings

An Accessibility settings panel was designed and implemented, accessible via the account icon in the navigation bar. The panel allows users to adjust text size scaling, which increases or decreases font sizes across the website without breaking layouts. The dark/light mode toggle was linked to the accessibility settings as well as remaining available directly in the navigation bar. All interactive elements were made keyboard-navigable with visible focus rings styled in the Interactive Normal colour from the style guide, meeting WCAG accessibility standards. All product images and decorative images were given descriptive alt text. Colour contrast was reviewed across the website to ensure compliance with WCAG AA (minimum 4.5:1 contrast ratio). Cookies consent pop-up was also created, which appears on the first visit, storing the user's preference (Accept All or Decline All) so it does not reappear on subsequent visits unless cookies are cleared. The responsive design of the website was finalised and tested across different viewport sizes, with navigation collapsing to a mobile-friendly layout, product grids reflowing appropriately and all buttons meeting minimum tap target sizes of 44x44px. Code across all newly created and edited files was commented thoroughly to document what each section does.




```
@* Auditory accessibility settings - Text-to-Speech features *@
<div class="a11y-category">...

<!-- ── COGNITIVE ── -->
@* Cognitive accessibility settings - reading and focus aids *@
<div class="a11y-category">...

<!-- ── NEUROLOGICAL ── -->
@* Neurological accessibility settings - motion reduction and stimulus control *@
<div class="a11y-category">...

<!-- ── PHYSICAL ── -->
@* Physical accessibility settings - motor control and navigation aids *@
<div class="a11y-category">...

<!-- ── SPEECH ── -->
@* Speech accessibility settings - voice input and screen reader support *@
<div class="a11y-category">...

<!-- ── VISUAL ── -->
@* Visual accessibility settings - color and contrast adjustments *@
<div class="a11y-category">
    <button class="a11y-category__trigger" aria-expanded="false" aria-controls="a11y-cat-visual">
        <span class="a11y-category__icon a11y-category__icon--visual" aria-hidden="true">◎</span>
        <span class="a11y-category__label">Visual <span class="a11y-category__badge">Colour &amp; Contrast</span><
        <svg class="a11y-category__chevron" width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="current
            <polyline points="6 9 12 15 18 9" />
        </svg>
    </button>
    <div id="a11y-cat-visual" class="a11y-category__content" hidden>
        @* Color mode selector - high contrast, grayscale, or inverted colors *@
        <div class="a11y-setting">
            <div class="a11y-setting__info">
                <span class="a11y-setting__label">Colour mode</span>
                <span class="a11y-setting__desc">Adjust colour rendering to improve readability</span>
            </div>
            <select id="a11y-colour-mode" class="a11y-select" aria-label="Colour mode">
                <option value="">Default</option>
                <option value="a11y-high-contrast">High contrast</option> @* Maximum contrast for low vision *@
                <option value="a11y-grayscale">Greyscale</option> @* Removes all color for color-blind users *@
                <option value="a11y-invert">Invert colours</option> @* Dark becomes light and vice versa *@
            </select>
        </div>
        @* Color saturation adjustment for visual sensitivity *@
        <div class="a11y-setting">
            <div class="a11y-setting__info">
                <span class="a11y-setting__label">Colour saturation</span>
                <span class="a11y-setting__desc">Reduce saturation if colours feel overwhelming; boost for low-vis
            </div>
            <select id="a11y-saturation" class="a11y-select" aria-label="Colour saturation">
```